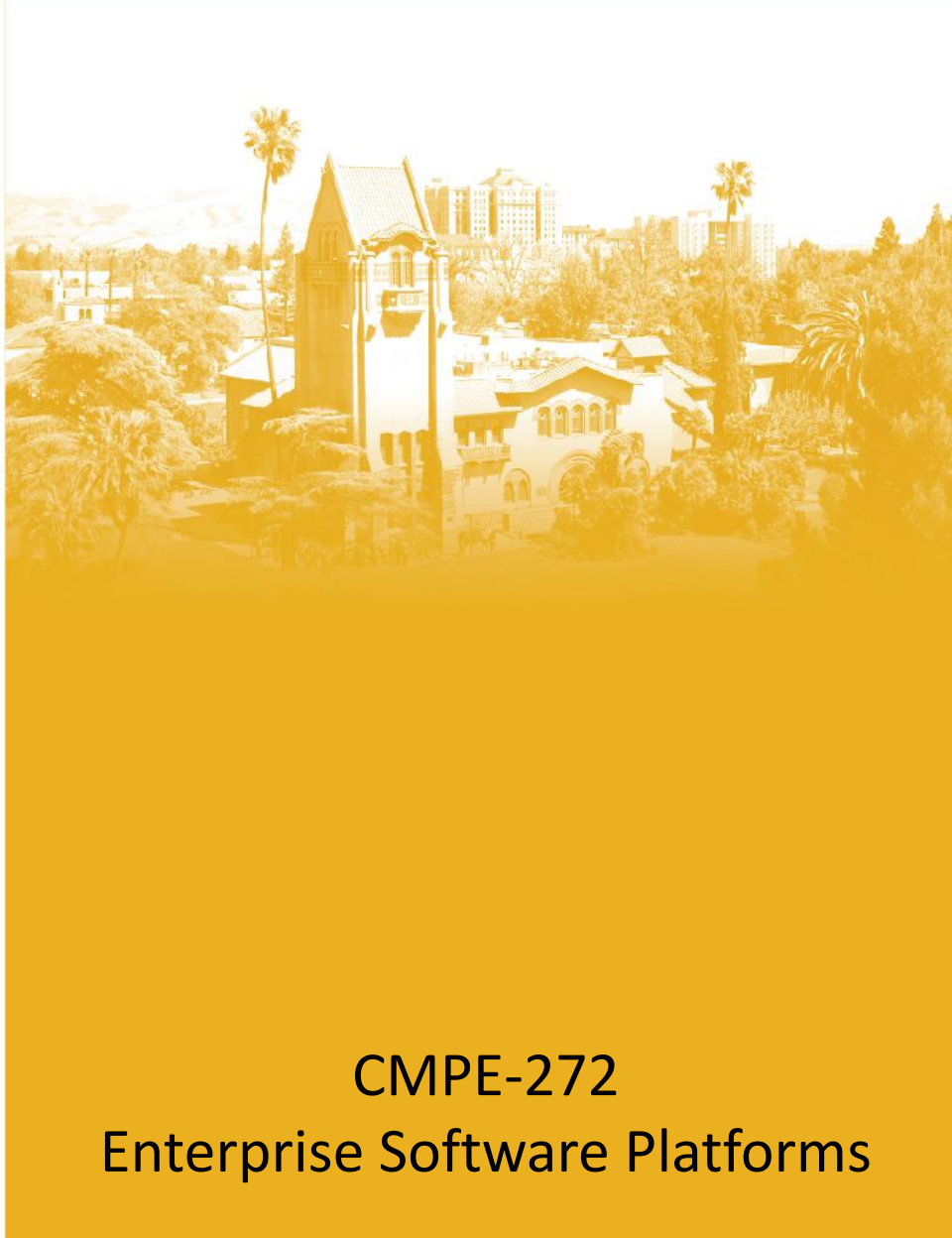




CMPE-272
Enterprise Software Platforms

Enterprise Software Platforms

Instructor: Andrew Bond

IT News: delivery pipelines, September 2026

- **GitHub Actions**, Sep 3: a REST API that reports when a runner version stops being supported, a read-only Dependabot-alerts permission for GITHUB_TOKEN, and job context fields that let a reusable workflow name its own source. Self-hosted runners below the minimum version stop running jobs from Sep 25.
- **Jenkins 2.568.3 LTS**, Sep 2: Java 21 or 25 required since the 2.555 line in April. Java 17 no longer runs it. For tonight's install, use Java 21.
- **SonarQube Server 2026.4**, July 22: an embedded MCP server so coding agents can query the quality gate, plus architecture analysis of the codebase. Static analysis is now read by agents as well as people.
- **DORA 2025 report**, nearly 5,000 respondents: 90% use AI at work; AI raises throughput and also raises instability; platform quality decides whether AI helps at all.
- **Gartner**: 80% of large engineering organisations run platform teams in 2026, up from 45% in 2022.

UNIT 4 · PLATFORM ENGINEERING AND THE DELIVERY LIFECYCLE (WEEK 5)

FA26 Update: From SDLC to Platform Engineering

- This unit pairs classic SDLC with platform engineering — where enterprise delivery effort concentrates today
- DevOps asked every team to own its pipeline; at scale that became 100 teams x 100 snowflake pipelines
- Platform engineering: a product team builds the paved road so stream-aligned teams can just ship
- Vocabulary for this unit: IDP, golden path, paved road, platform-as-product, DevEx

Internal Developer Platforms (IDPs)

- Self-service portal + service catalog + templates + environments on demand
- Backstage (Spotify -> CNCF), Port, Cortex — and build-vs-buy is a genuine decision
- **Golden path:** scaffold -> repo -> CI/CD -> deploy -> observability, pre-wired to policy
- The platform is an internal product with users, a roadmap and SLAs — not a ticket queue

Measuring Delivery: DORA Metrics

- Deployment frequency · Lead time for changes · Change-failure rate · Time to restore
- Elite vs low performers: deploy on demand vs monthly; restore in under an hour vs over a week
- Use them as team health signals, not individual performance reviews (Goodhart's law applies)
- SPACE framework complements DORA: satisfaction, performance, activity, communication, efficiency
- DORA 2025: 90% of nearly 5,000 respondents use AI; throughput up, instability up; platform quality decides whether AI helps

Governance Through Code

- Policy-as-code: OPA/Rego, Kyverno — rules enforced in the pipeline, not in review meetings
- Production-readiness scorecards as code: security, observability, ownership checks
- Compliance evidence generated by the platform itself: SBOMs, provenance, audit trails
- **Preview:** software supply-chain security returns in Unit 11 (Week 13)

Agenda

- Software Development Lifecycle (SDLC)
- DevOps & Process Frameworks
- Methodologies
- Tools
- Q&A

SOFTWARE DEVELOPMENT LIFECYCLE (SDLC)

What is SDLC?

- A systems development life cycle is composed of a number of clearly defined and distinct work phases
- Contract between Stakeholders and IT on the process that will be followed for developing solutions

Why do we need SDLC?

- To bring consistency across project teams in terms of process – common process ‘Language’
- To make solution development process repeatable
- To ensure teams understand deliverables from each stage.
- To ensure teams are following same templates for deliverables

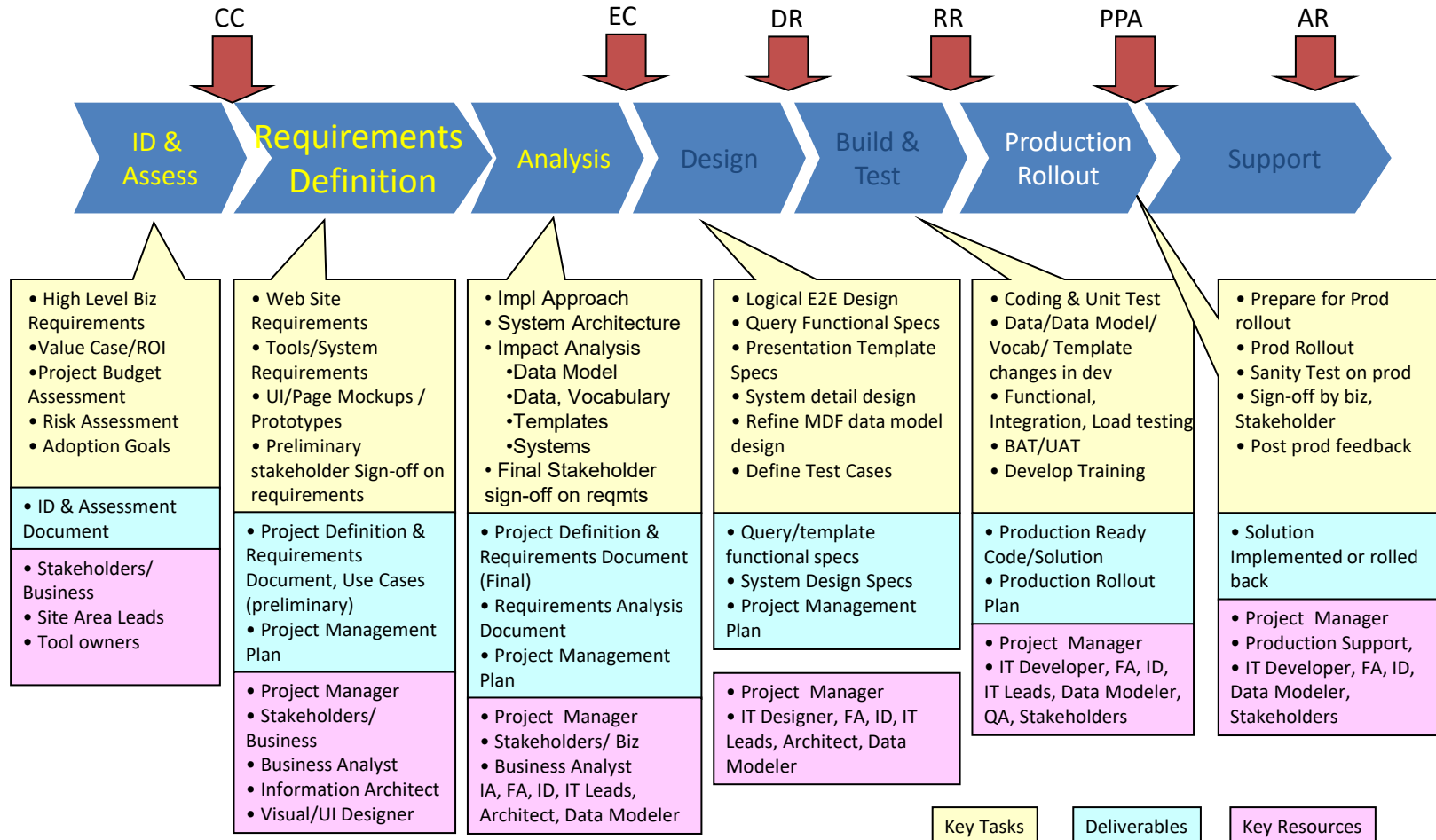
How will it change what we do today?

- We still implicitly follow a process. Following a formal SDLC will make process steps explicit, clear and consistent
- Will incorporate ROI, checks and balances for projects. If the cost justification isn't there, then project will be canceled
- Some of the tasks that we are doing in later stages will be required to be done in the initial stages. This will help identifying issues early in the life cycle.
- Will have more formal planning & tracking for major milestones like 'Readiness Review' rather than leaving it for the last minute
- Will have some impact on our timelines but –
 - Will help us in developing right solutions first time itself
 - Will help in more efficient project planning and management
- The idea is not to make process very heavy and documentation centric – but to document information consistently in right documents (and not in emails and people's head).

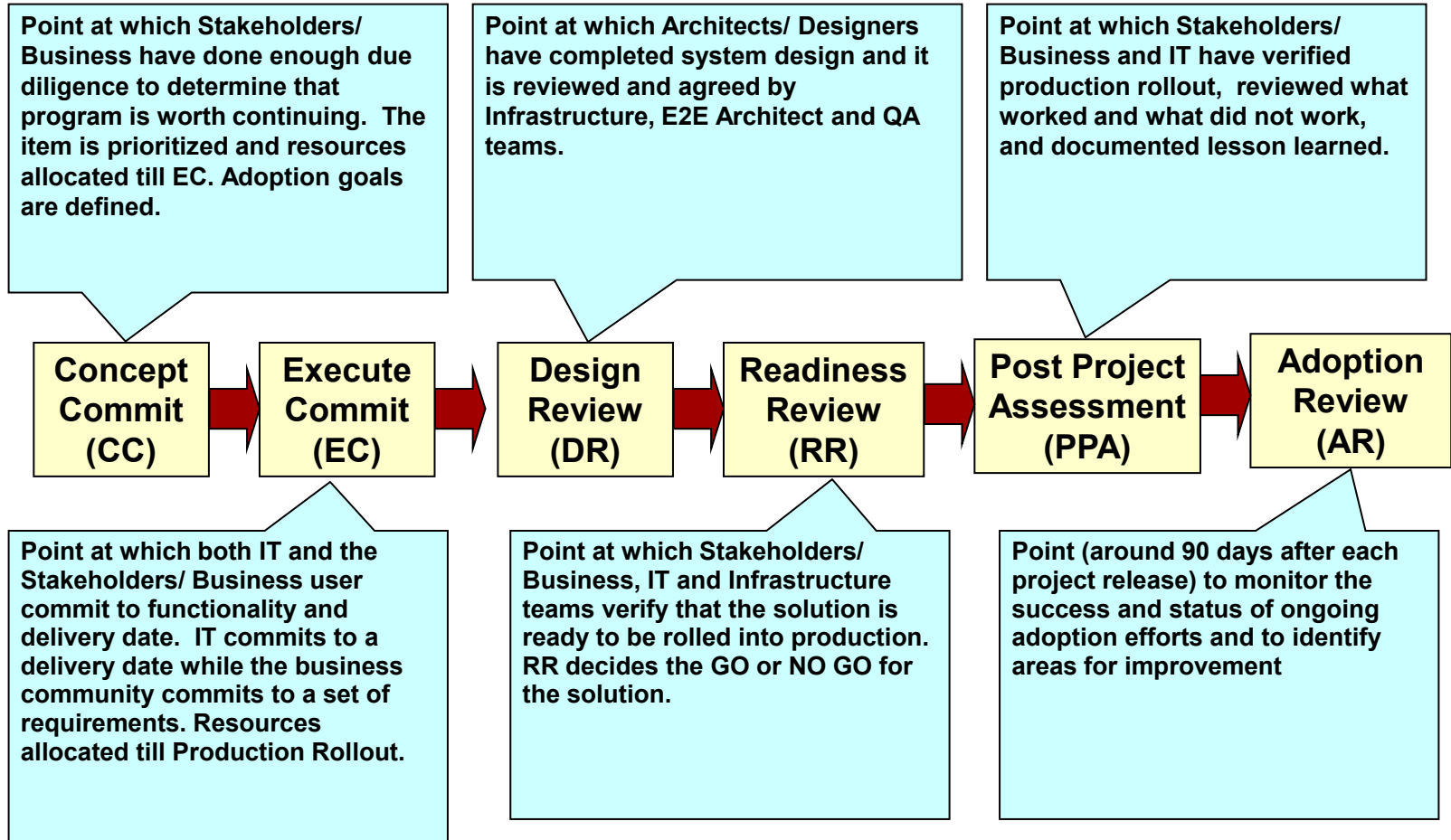
What SDLC covers/Does NOT cover?

- Covers life cycle for an enhancement request/New Application once the pre-assessment (Business Commit) for the request is completed
- Does NOT cover Enhancement Requests Consolidation and Prioritization Process
- Does NOT cover process for small projects (a.k.a. operational items)

SDLC Stages - Overview



SDLC Major Milestones - Overview



Popular SDLC models

- **Waterfall**
 - a linear [sequential](#) (non-iterative) [design](#) approach for [software development](#), in which progress flows in one direction downwards (like a [waterfall](#)) through the phases of conception, initiation, [analysis](#), [design](#), [construction](#), [testing](#), [deployment](#) and [maintenance](#)
- **Spiral**
 - A risk-driven [process model](#) generator for software projects. Based on the unique risk patterns of a given project, the spiral model guides a team to adopt elements of one or more process models, such as [incremental](#), [waterfall](#), or [evolutionary prototyping](#)
- **Agile software development**
 - requirements and solutions evolve through the collaborative effort of self-organizing [cross-functional teams](#).^[1] It advocates adaptive planning, evolutionary development, early delivery, and continuous improvement, and it encourages rapid and flexible response to change
- **Rapid prototyping**
 - After preliminary requirements gathering is accomplished, a simple working model of the system is constructed to visually show the users what their requirements may look like when they are implemented into a finished system
- **Iterative and incremental development**
 - any combination of both [iterative design](#) or [iterative method](#) and [incremental build model](#) for software development
- **Synchronize and Stabilize**
 - (sometimes just called sync-and-stabilize) is a systems development life cycle model in which teams work in parallel on individual application modules, frequently synchronizing their code with that of other teams, and debugging (stabilizing) code regularly throughout the development process

DEVOPS & PROCESS FRAMEWORKS

Introduction to DevOps

What is DevOps?

In a nutshell:

End-to-end Release = Conception + Development/test + Operations

DevOps = Streamlined, rapid, repeatable process of end-to-end release

- DevOps = Development + Operations
- It is the latest trend in software development and delivery, which focus on partnership or collaboration between developers, business, and operations staff, coupled with automation tools for an end-to-end streamlined, rapid, and repeatable software release cycle through the SDLC.



Business

Setup requirements in product backlogs, led by PO and Scrum masters.

Development Phase

After assigning stories for development, team tests, build, package applications.

Operations

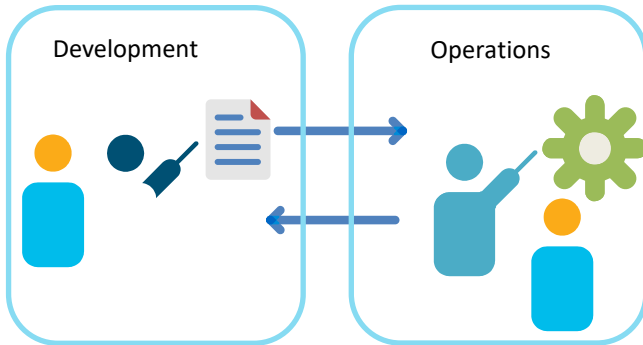
Operations and Infrastructure team provision to upper environments and monitor applications, support, secure.

Introduction to DevOps

DevOps is a software development methodology that focuses on close collaboration between software development, tests, and operations teams, to optimize and automate the whole SDLC process.

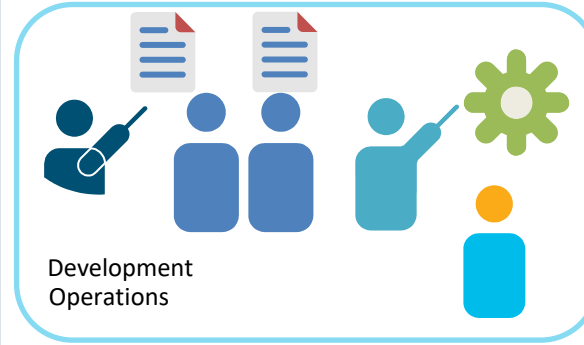
Before DevOps rollout

The teams were working in isolation or silos with clear demarcation of tasks and without integration or orchestration.



After DevOps

Development, tests, system admin, and operations teams work together in collaboration as a whole team.



DevOps Benefits

DevOps breaks silos base teams that exists in traditional IT, helping organizations to rapidly produce and release software products and sevicees with better quality, faster speed, less defects, and enabling better collaboration and automaton of the whole SDLC phases.

DevOps Principles

Principles	Description
Cohesive Teams 	• Strong collaboration between business, developers, testers, and other IT teams, required for smooth release of small and frequent changes.
Automate everything 	• Process of releasing software must be repeatable and reliable through automation of tasks involved in releasing the software.
Strong source control 	• A strong version control process not only for source code, but also for infrastructure as code and other artifacts. Following a naming standard for branches and artifacts to enable RPA and orchestration of tasks, is recommended.
Test early and frequent 	• Environment configuration and infrastructure must be tested early and frequently, during the software life cycle.
Improve continuously 	• Teams must consistently evaluate and update their practices, process, tools, metrics, environments and application properties, as well as source and binary scanning procedures.

DevOps Technologies

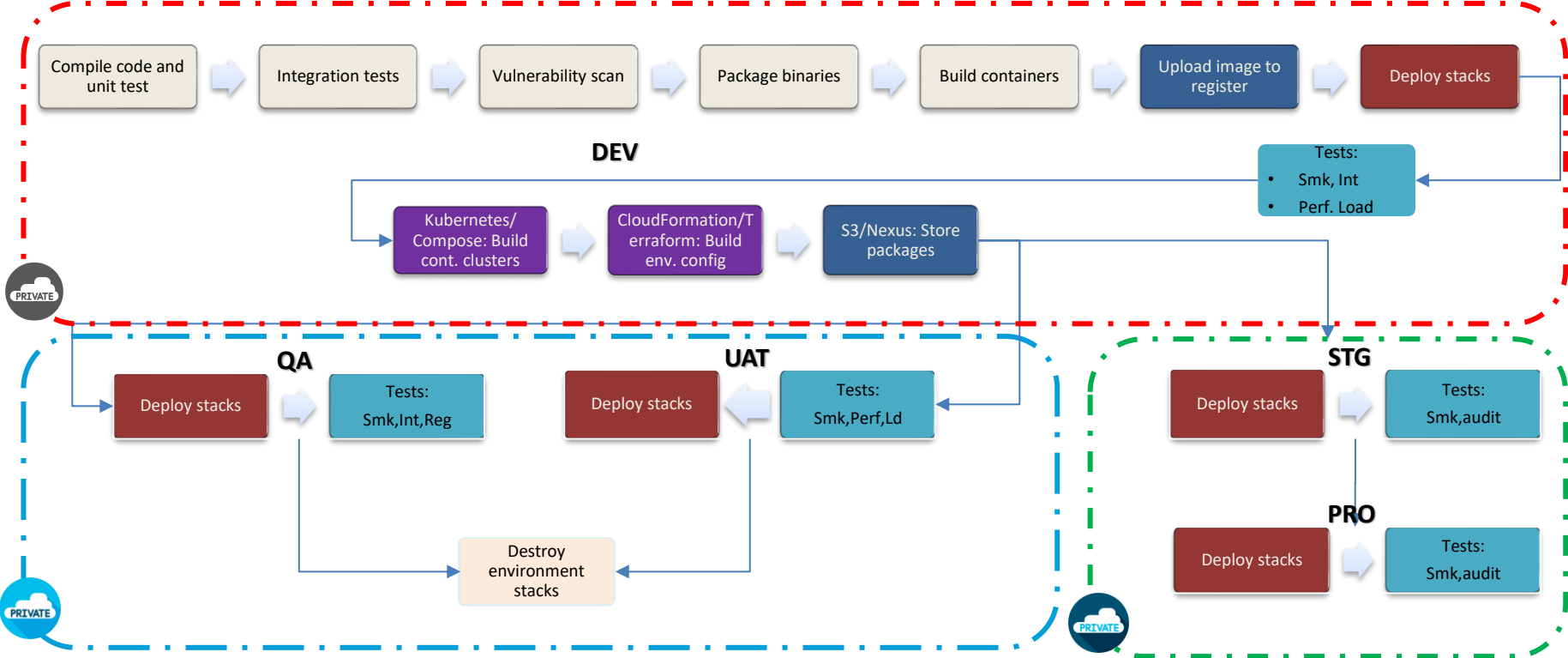
- DevOps embrace Agile approach to enable continuous integration (CI), Continuous Testing, Continuous Delivery, and Continuous Deployment.
- DevOps focus on building a culture of collaboration to mitigate issues and resolve complexity.
- Works in the sense of orchestration to create automated delivery pipelines.
- DevOps embrace migration to the cloud leveraging PaaS virtualized environments and containerization of micro services to make a more robust, scalable, available, and resilient applications and environments.
 - SaaS based organizations becomes operators of their software (e.g. Salesforce) services, that requires to meet complex environments and Service Level Agreement (SLA). This complexity limits the ability to remain competitive when using on premise datacenters; therefore, the way around is to move to the cloud.
 - The product does not end on shipping software; they have to support it, provide the environment, and maintain it.

Example: Large Enterprise CI/CD Pipelines & Orchestration

SOX GROUPS:

- Development
- Testing
- Operations

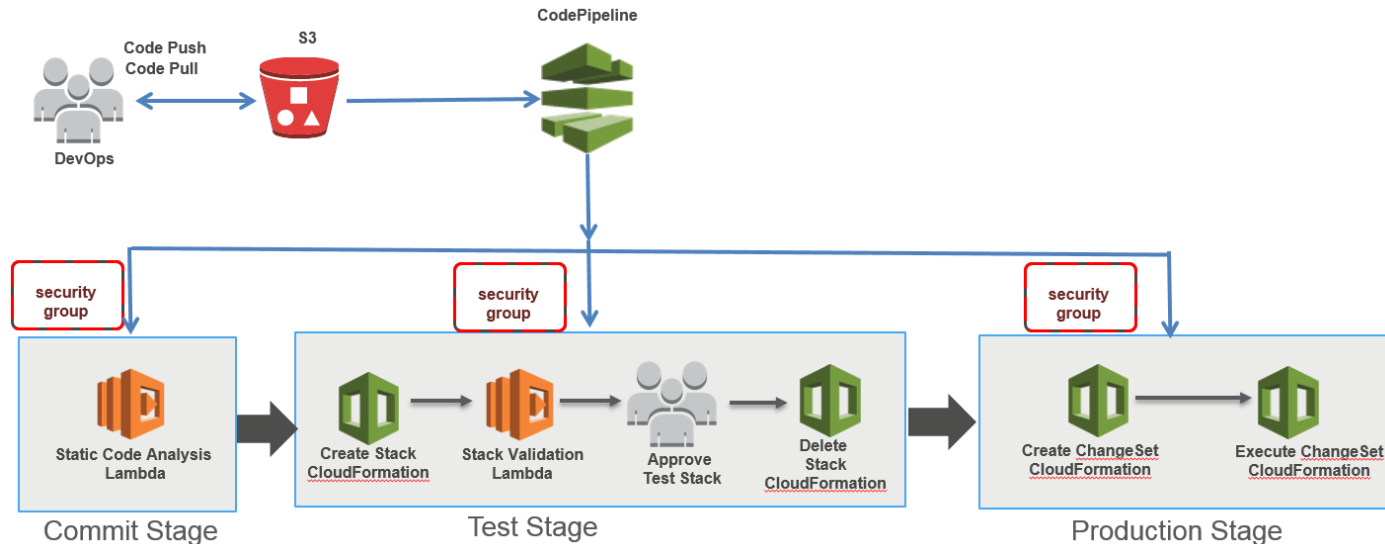
Continuous Integration Stages in CI/CD pipelines



DEvelopment, **QA**uality Assurance, **UA**ser Acceptance Testing, **STG**(Staging), **PRO**duction

Compliance as Code

- [DevOps Audit Defense Toolkit](#)
- [Enforce regulatory compliance controls for PCI, HIPAA, GDPR, and beyond, across the cloud-native application lifecycle](#)
- [Implementing DevSecOps Using AWS CodePipeline](#)

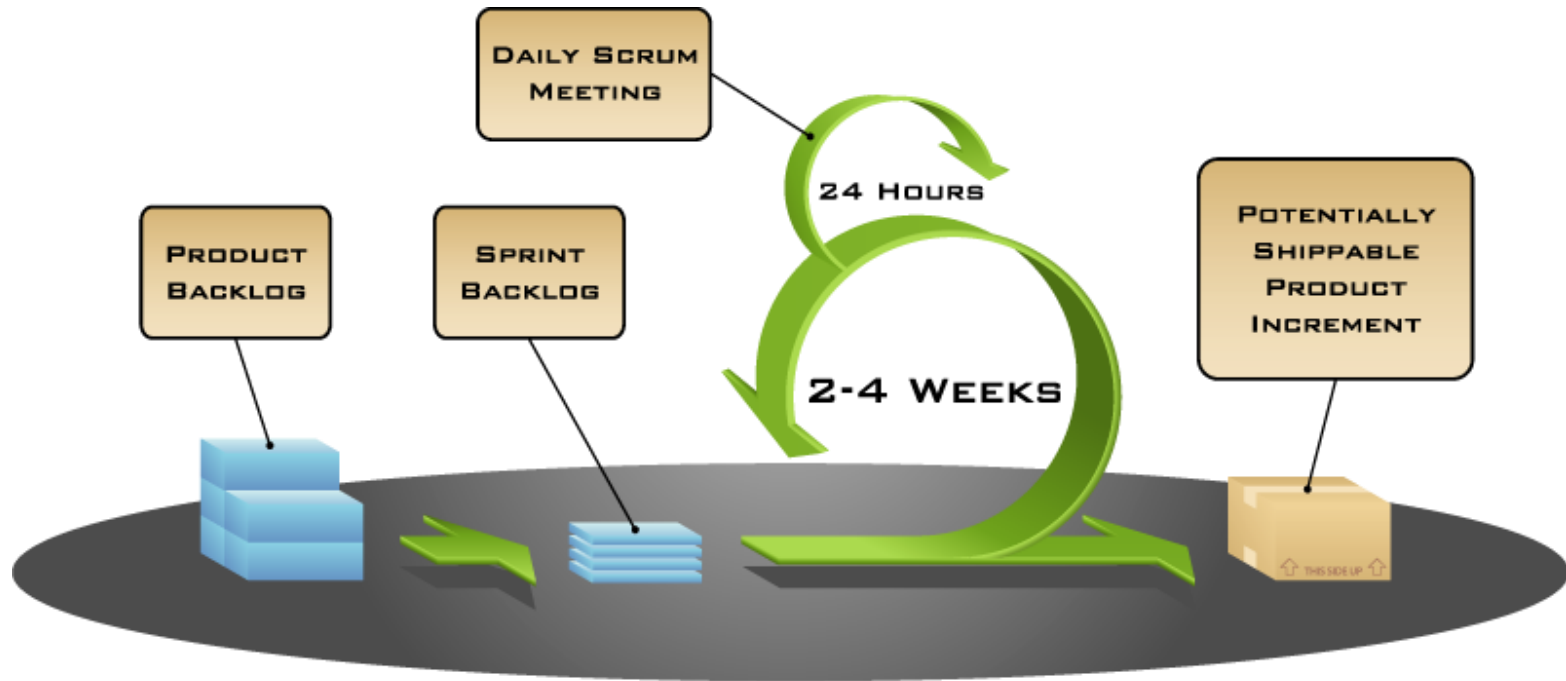


METHODOLOGY - SCRUM

Scrum Basics

- Scrum is an agile process that allows us to focus on delivering the highest business value in the shortest time.
- The business sets the priorities. Teams self-organize to determine the best way to deliver the highest priority features.
- Projects progress via a series of **sprints**. In keeping with an agile methodology, sprints are **timeboxed** to no more than a month long, most commonly two weeks
 - A planning meeting starts the sprint – team members figure out how many items they can commit to, and then create a sprint **backlog** – a list of the tasks to perform during the sprint
 - During an agile Scrum sprint, the Scrum team takes a small set of features from idea to coded and tested functionality. At the end, these features are done, meaning coded, tested and integrated into the evolving product or system
 - On each day of the sprint, all team members should attend a daily Scrum meeting, including the ScrumMaster and the product owner. This meeting is timeboxed to no more than 15 minutes.

Putting it all together



Sequential vs. Overlapping Development

Requirements

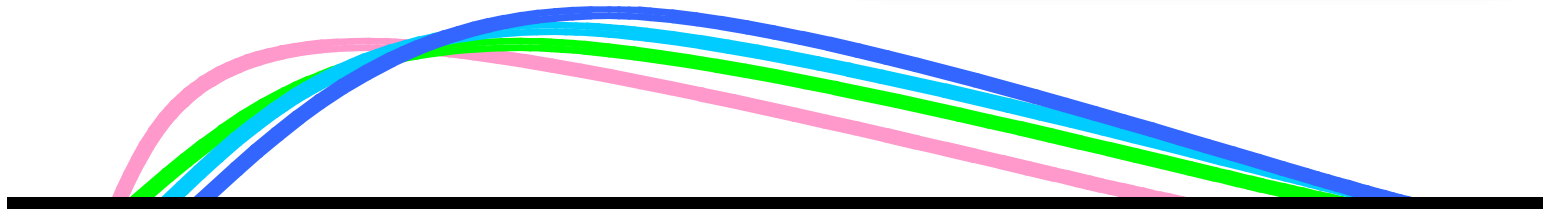
Design

Code

Test

Rather than doing all of one thing
at a time...

...Scrum teams do a little of
everything all the time



Source: "The New New Product Development Game" by Takeuchi and Nonaka. *Harvard Business Review*, January 1986.

METHODOLOGY - KANBAN

Kanban for lean manufacturing

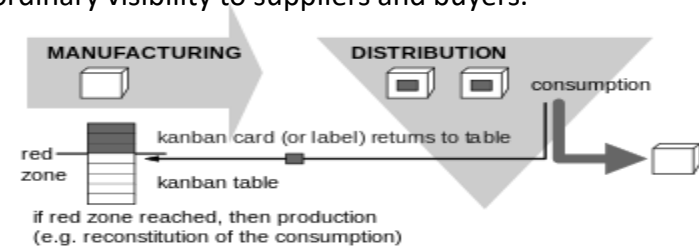
Kanban (看板) (signboard or billboard in Japanese)

TPS (Toyota Production System) invented “kanban”

- In Lean Manufacturing, a kanban is a signaling device that gives authorization and instructions for production or withdrawal (conveyance) of items in a pull system.

Kanbans maintain inventory levels; a signal is sent to produce and deliver a new shipment as material is consumed. These signals are tracked through the replenishment cycle and bring extraordinary visibility to suppliers and buyers.

Purpose:	Logistic control system
Implemented at:	Toyota
Date implemented:	1953



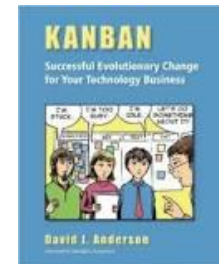
Toyota's Six Rules from TPS

1. Later process picks up the number of items indicated by the kanban at the earlier process.
2. Earlier process produces items in the quantity and sequence indicated by the kanban.
3. No items are made or transported without a kanban.
4. Always attach a kanban to the goods.
5. Defective products are not sent on to the subsequent process.
The result is 100% defect-free goods.
6. Reducing the number of kanban increases the sensitivity.

Kanban for Software Development

- David J Anderson created “K”anban, now known as Kanban Method. Kanban for knowledge work is a management method of just-in-time delivery based on capacity to do work. As such, here the focus is not to overload the team members.
- In Kanban, the process workflow, from definition of work to its delivery to the customer, is displayed for participants to see and team members pull work from each queue.
- Kanban for software development means a visual process management system that tells what to produce, when to produce it, and how much to produce- inspired by the Toyota Production System and Lean Manufacturing.
- For more details on “Kanban (Method)”, please check out David J. Anderson’s website: <http://www.djaa.com/principles-kanban-method-0> and his excellent book:
Kanban: Successful Evolutionary Change for Your Technology Business
by David J. Anderson, Publisher: Blue Hole Press (April 7, 2010)

<https://www.youtube.com/watch?v=R8dYLBjITUE> (4 mins)



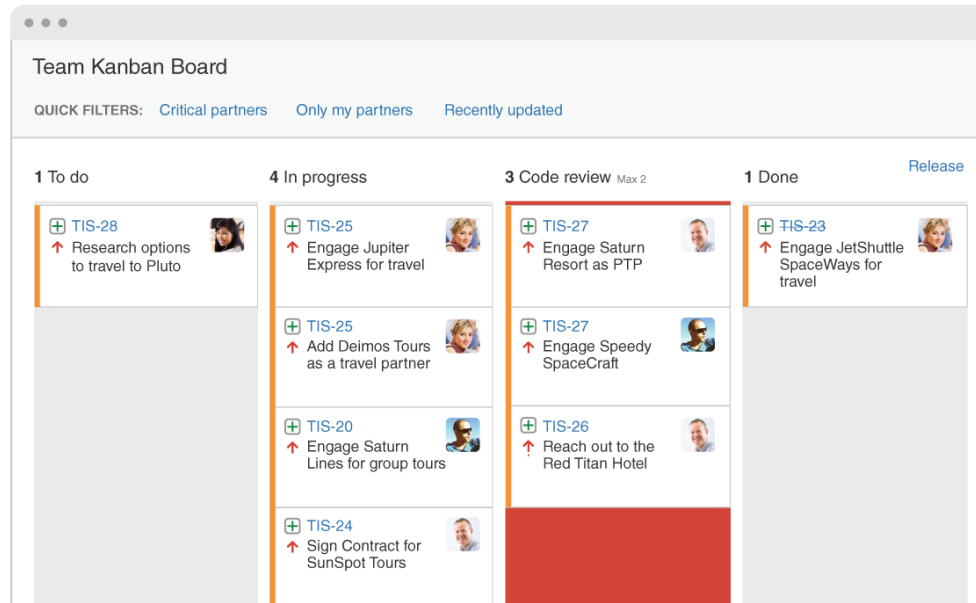
Kanban Values / Philosophy

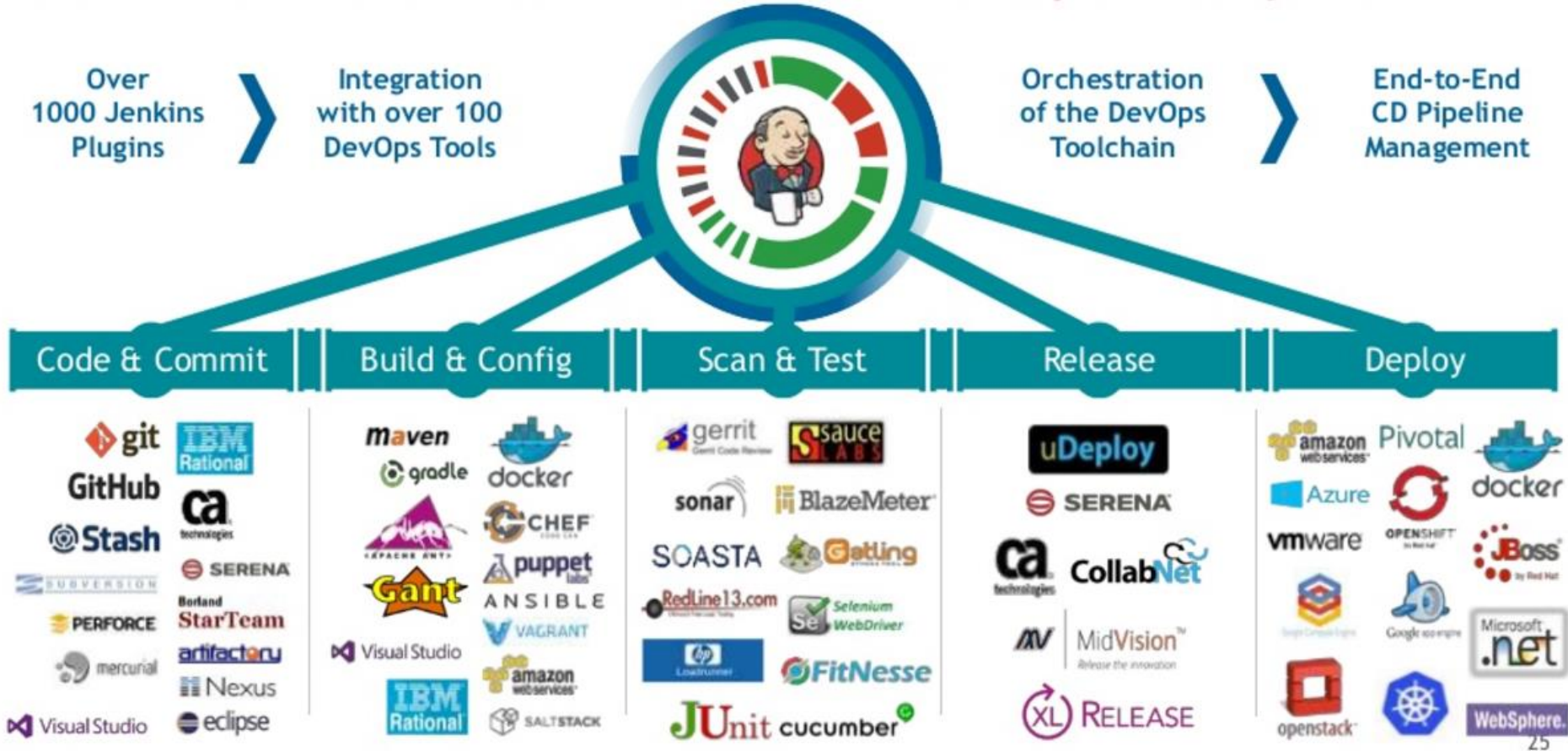
合作

- **Transparency** - sharing information openly using clear and straightforward language improves the flow of business value
- **Balance** - different aspects, viewpoints, and capabilities must be balanced in order to achieve effectiveness
- **Collaboration** - Kanban was created to improve the way people work together
- **Customer Focus** - Kanban systems aim to optimize the flow of value to customers that are external from the system but may be internal or external to the organization in which the system exists
- **Flow** - Work is a continuous or episodic flow of value
- **Leadership** - Leadership (the ability to inspire others to act via example, words, and reflection) is needed at all levels in order to realize continuous improvement and deliver value.
- **Understanding** - Individual and organizational self-knowledge of the starting point is necessary to move forward and improve.
- **Agreement** - Everyone involved with a system are committed to improvement and agree to jointly move toward goals while respecting and accommodating differences of opinion and approach.
- **Respect** - Value, understand, and show consideration for people.

Kanban boards – 看板

- The work of all kanban teams revolves around a kanban board, a tool used to visualize work and optimize the flow of the work among the team
- A basic kanban board has a three-step workflow: To Do, In Progress, and Done
- The kanban methodology relies upon full transparency of work and real-time communication of capacity, therefore the kanban board should be seen as the single source of truth for the team's work





DEVOPS AUTOMATION WITH JENKINS

What's Jenkins/Hudson



1. Jenkins/Hudson is:

- The leading open source automation server, Jenkins provides hundreds of plugins to support building, deploying and automating any project.
- Open source continuous integration tool written in Java. The project was [forked from Hudson after a dispute with Oracle](#). Jenkins provides continuous integration services for software development. It is a server-based system running in a servlet container such as Apache Tomcat.
[https://en.wikipedia.org/wiki/Jenkins_\(software\)](https://en.wikipedia.org/wiki/Jenkins_(software))
- An [award-winning](#), cross-platform, **continuous integration and continuous delivery** application that increases your productivity. Use Jenkins to **build and test your software projects continuously** making it easier for developers to integrate changes to the project, and making it easier for users to obtain a fresh build. It also allows you to **continuously deliver** your software by providing powerful ways to define your build pipelines and integrating with a large number of testing and deployment technologies



2. [Plug-in](#) extensibility (Over 1500+ plugins)

3. [MIT license](#)



- Without Jenkins
 - Commit code
 - Project manager should ask programmer for latest version
 - Programmer asks tester to put the files on the dev server
 - Test the programmer and show to the project manager
- With Jenkins
 - Writing the code and committing it.
 - 5 minutes later the code is on dev environment without anyone having to do anything.
 - The project manager can look at the dev environment knowing it has the latest work.
 - When something breaks you can look at the dev environment to check the latest version.
 - When the project manager is happy with the build on dev, they click a button to deploy it to staging.



Jenkins

- Automated testing
 - Scheduling building
- Notification:
 - email notification for a build project.
- Reporting
 - Reporting plugins such as Junit
- Code Analysis
 - Code Analysis plugin. The various plugins can be found at <https://plugins.jenkins.io/warnings-ng/>
- Distributed Builds
 - Jenkins agent machines



Jenkins

- Automated Deployment
 - plugins available to transfer the build files after a successful build to the respective application/web server
- Metrics & Trends
 - plugin available to calculate the following metrics for all of the builds once installed
 - Mean Time To Failure (MTTF)
 - Mean Time To Recovery (MTTR)
 - Standard Deviation of Build Times
- Server Maintenance
 - Shutdown, Restart, reload the configuration, Backup
- Security:
 - having the ability to setup users and their relevant permissions on the Jenkins instance
- Remote Testing
 - tests can be run on remote agent machines via the controller and agent

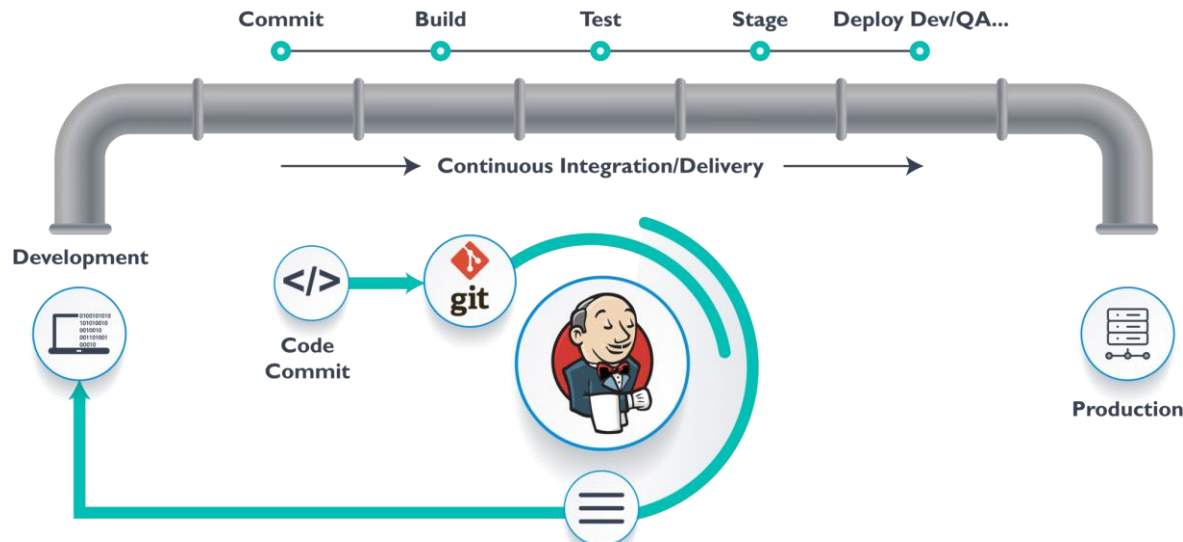
Jenkins Pipeline



Introducing Jenkins
Blue Ocean



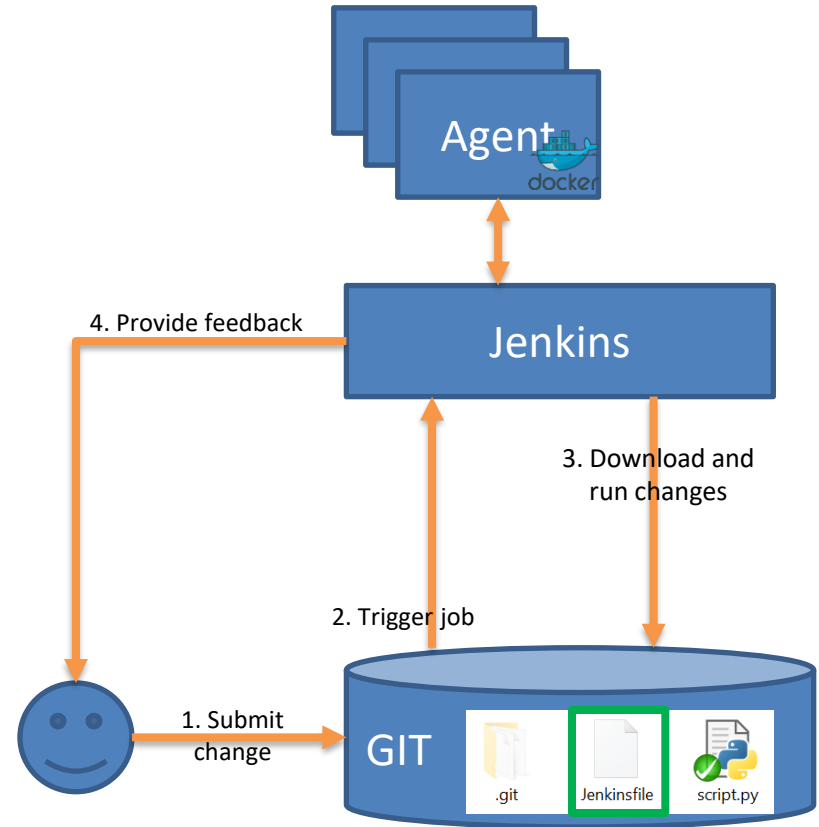
- Jenkins Pipeline (or simply "Pipeline") is a suite of plugins which supports implementing and integrating *continuous delivery pipelines* into Jenkins
- Jenkins Pipeline provides an extensible set of tools for modeling simple-to-complex delivery pipelines "as code"
- The definition of a Jenkins Pipeline is typically written into a text file (called a Jenkinsfile) which in turn is checked into a project's source control repository
- **Jenkins Blue Ocean** (redesign the user experience of Jenkins)
 - **Sophisticated visualizations** of continuous delivery (CD) Pipelines, allowing for fast and intuitive comprehension of Pipeline status.
 - **Pipeline editor** - makes creation of Pipelines approachable by guiding the user through an intuitive and visual process to create a Pipeline.
 - **Personalization** to suit the role-based needs of each member of the team.
 - **Pinpoint precision** when intervention is needed and/or issues arise. Blue Ocean shows where in the pipeline attention is needed, facilitating exception handling and increasing productivity.
 - **Native integration for branch and pull requests**, enables maximum developer productivity when collaborating on code with others in GitHub and Bitbucket.



https://www.youtube.com/watch?v=k_fVIU1FwP4 (1 min)

Pipeline as code

- With **Blue Ocean**, Job/Pipeline definition is also stored in SCM
- **Sophisticated visualizations** of continuous delivery (CD) Pipelines, allowing for fast and intuitive comprehension of your Pipeline's status
- **Pipeline editor** - makes creation of Pipelines approachable by guiding the user through an intuitive and visual process to create a Pipeline
- **Personalization** to suit the role-based needs of each member of the team
- **Native integration for branch and pull requests**
- Pipeline can run in a dedicated Docker container
- Each Git branch will automatically have its own job



Jenkins Plugins

- Build triggers
- Source code management
- Build tools
- Build wrappers
- Build notifiers
- Build reports
- Artifact uploaders
- UI plugins
- Authentication and user management



Categories

[Platforms](#)

[User interface](#)

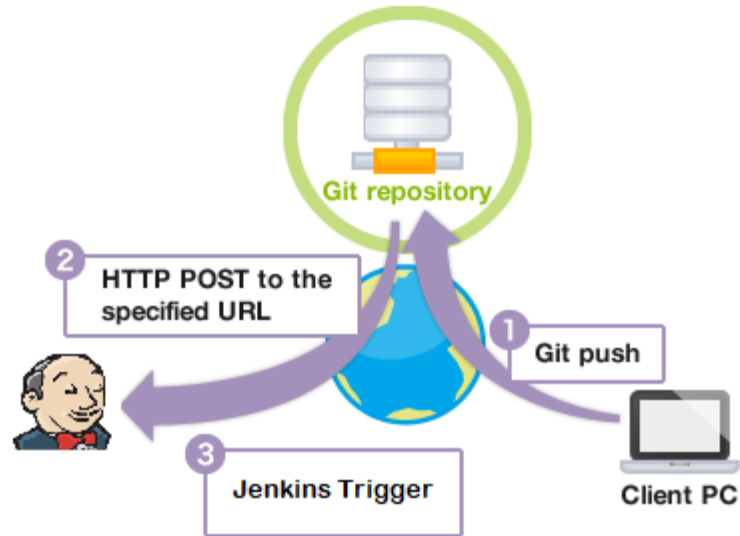
[Administration](#)

[Source code management](#)

[Build management](#)

Build Trigger

- Manually click build button
- Build periodically
- Build whenever a SNAPSHOT dependency is built
- Build after other projects are built
- Poll SCM
- IRC, Jabber, ...
- [WebHooks](#) (an HTTP callback: a simple event-notification via HTTP POST)



Build Tools Supported in Jenkins

- Java
 - Maven (built-in), Ant, Gradle
- .Net
 - MSBuild, PowerShell
- Shell script
 - Python, Ruby, Groovy
- Python
 - [PyInstaller](#)
- Node.js, React
 - [Npm](#)

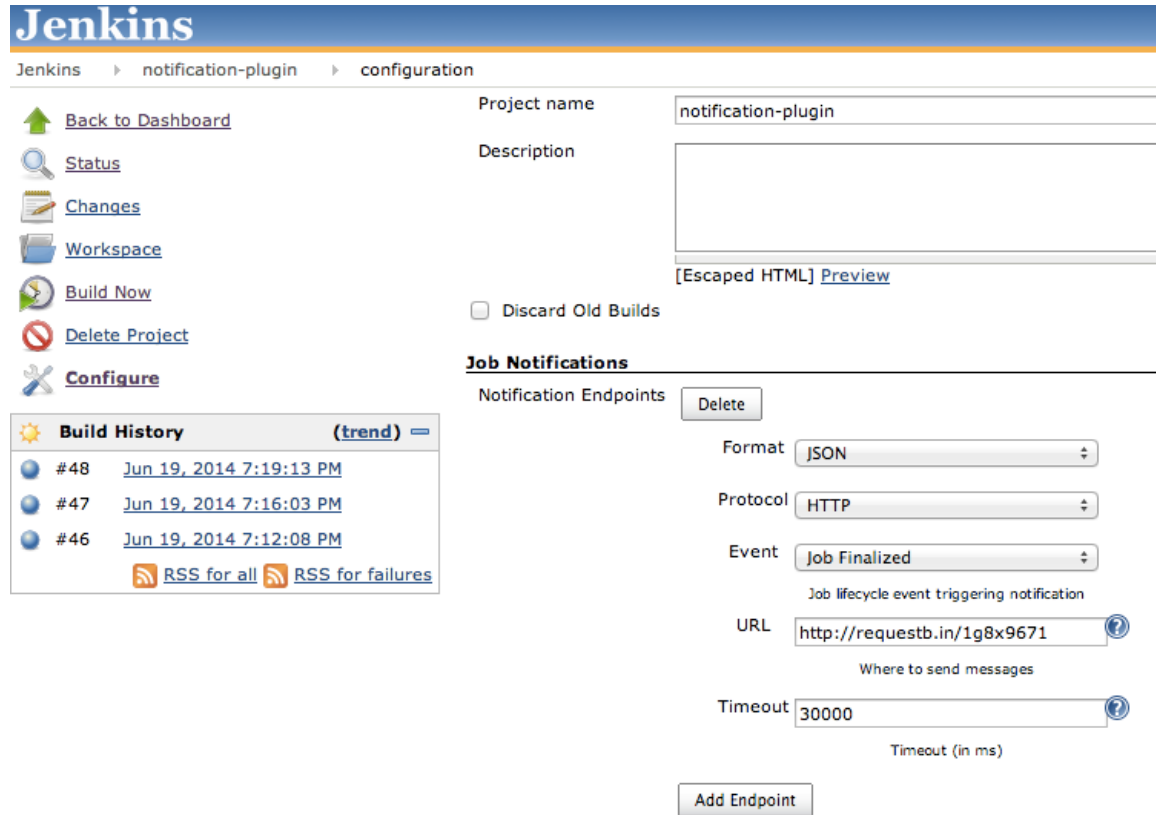
Build Wrapper

- [Wrappers](#) alter the way the build is run as well as the build output
- Build name (version no) setter
- Virtual machine (VMWare, Virtual Box)
- Set environment variable
- ClearCase release plugin
- ...

Build/Job Notification

- Notification Plugins

- E-mail
- Twitter
- Jabber
- IRC
- RSS
- Google calendar



The screenshot shows the Jenkins web interface for configuring a notification plugin. The breadcrumb trail is "Jenkins > notification-plugin > configuration". On the left sidebar, there are links: "Back to Dashboard", "Status", "Changes", "Workspace", "Build Now", "Delete Project", and "Configure". Below these is a "Build History" section showing three builds: #48 (Jun 19, 2014 7:19:13 PM), #47 (Jun 19, 2014 7:16:03 PM), and #46 (Jun 19, 2014 7:12:08 PM), with links for "RSS for all" and "RSS for failures". The main configuration area has a "Project name" field set to "notification-plugin" and a "Description" text area. Below these is a "Discard Old Builds" checkbox. The "Job Notifications" section contains a "Notification Endpoints" table with one entry. The entry has a "Delete" button, a "Format" dropdown set to "JSON", a "Protocol" dropdown set to "HTTP", an "Event" dropdown set to "Job Finalized", a "URL" field set to "http://requestb.in/1g8x9671", and a "Timeout" field set to "30000". A "Where to send messages" label is positioned between the URL and timeout fields. At the bottom right is an "Add Endpoint" button.

Jenkins

Jenkins > notification-plugin > configuration

[Back to Dashboard](#)

[Status](#)

[Changes](#)

[Workspace](#)

[Build Now](#)

[Delete Project](#)

[Configure](#)

Build History (trend) —

#48	Jun 19, 2014 7:19:13 PM
#47	Jun 19, 2014 7:16:03 PM
#46	Jun 19, 2014 7:12:08 PM

[RSS for all](#) [RSS for failures](#)

Project name: notification-plugin

Description:

☐ Discard Old Builds

Job Notifications

Notification Endpoints

Delete	Format: JSON	Protocol: HTTP	Event: Job Finalized	URL: http://requestb.in/1g8x9671	Timeout: 30000
--------	--------------	----------------	----------------------	----------------------------------	----------------

Job lifecycle event triggering notification

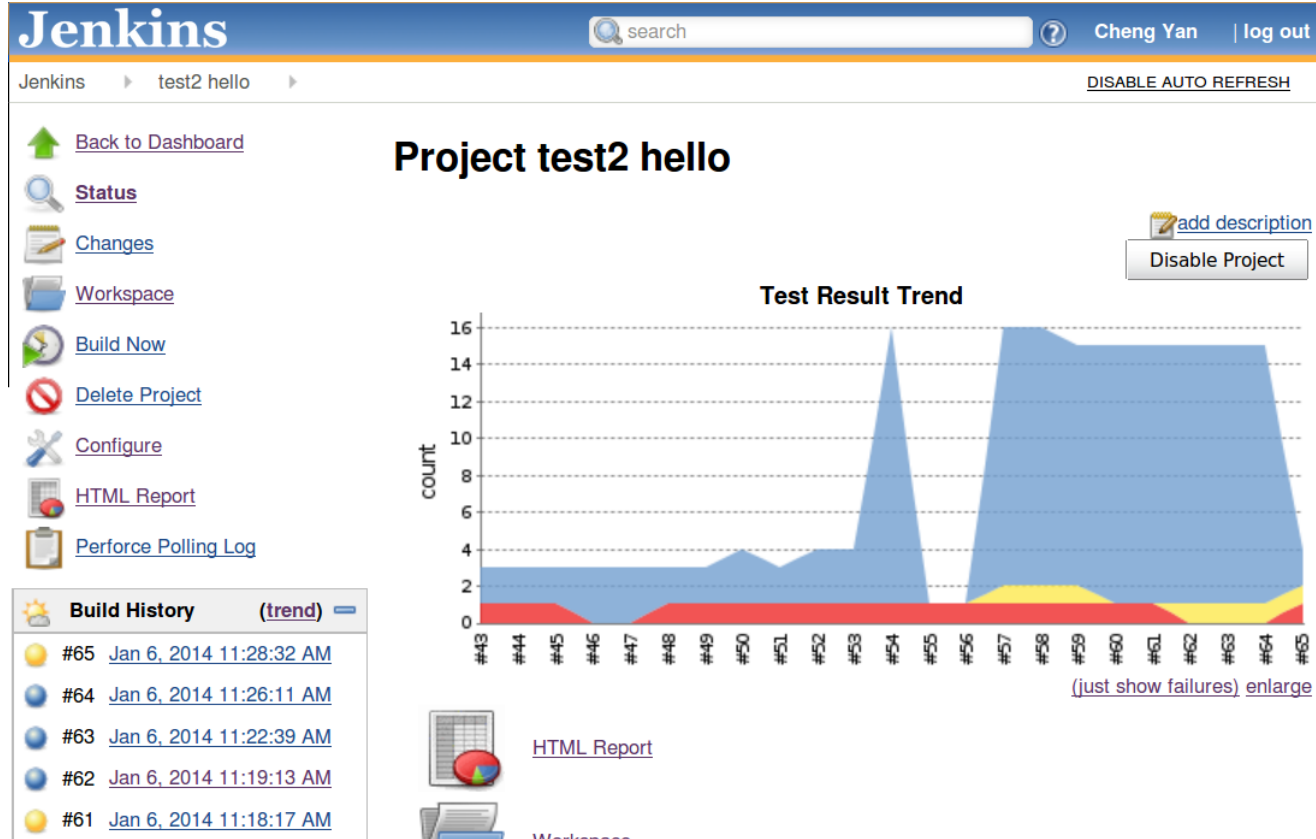
Where to send messages

Timeout (in ms)

Add Endpoint

Build Report

- Static Code Analysis
 - Checkstyle, PMD, SpotBugs, Compiler Warning
- Test Report & Code Coverage
 - JUnit, TestNG, JaCoCo, Cobertura
- Open Tasks
- Build History



Static Code Analysis

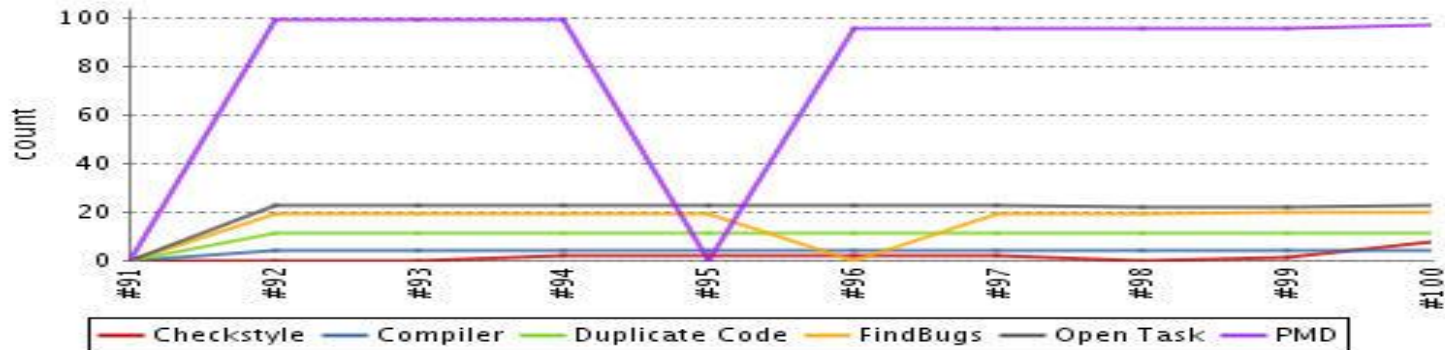
All

DashBoard

+

Warnings per Job

Job ↓	Checkstyle	Duplicate Code	FindBugs	PMD	Open Tasks	Compiler Warnings	Total
 M-Multi-Freestyle	14 	-	0	86 	24	0 	124
 M-Multi-Maven	14	11	12	80	0	4	121
 M-Single-Freestyle	0	0	15	18	99	3	135
 M-Single-Freestyle-Site	5967	2	-	18	5	0	5992
 M-Single-Maven	-	-	-	-	121	0	121
 M-Single-Maven-2	-	4	16	243	4	0	267
 M-Single-Maven-Site	1784	2	72	2	0	-	1860



CheckStyle

CheckStyle Result

Warnings Trend

All Warnings	New Warnings	Fixed Warnings
6766	2	0

Summary

Total	High Priority	Normal Priority	Low Priority
6766	6766	0	0

Details

Modules	Package	Files	Categories	Types	Warnings	Details	New
Type	Total	Distribution					
AvoidInlineConditionalsCheck	47						
AvoidNestedBlocksCheck	2						
AvoidStarImportCheck	1						
ConstantNameCheck	11						
DesignForExtensionCheck	789						
EmptyBlockCheck	29						
FinalClassCheck	5						
FinalParametersCheck	980						
HiddenFieldCheck	288						
HideUtilityClassConstructorCheck	9						
InnerAssignmentCheck	1						
InterfaceIsTypeCheck	8						
JavadocMethodCheck	1472						
JavadocPackageCheck	1						

FindBugs (now SpotBugs)

FindBugs Result

Warnings Trend

All Warnings	New this build	Fixed Warnings
22	0	0

Summary

Total	High Priority	Normal Priority	Low Priority
22	0	0	22

Details

Modules	Package	Files	Categories	Types	Warnings	Details
---------	---------	-------	------------	--------------	----------	---------

Type	Total	Distribution
CN_IMPLEMENTES_CLONE_BUT_NOT_CLONEABLE	1	
DLS_DEAD_LOCAL_STORE	1	
EI_EXPOSE_REP	4	
EI_EXPOSE_REP2	7	
EQ_CHECK_FOR_OPERAND_NOT_COMPATIBLE_WITH_THIS	3	
EQ_DOESNT_OVERRIDE_EQUALS	2	
ICAST_IDIV_CAST_TO_DOUBLE	1	
OBL_UNSATISFIED_OBLIGATION	1	
RV_RETURN_VALUE_IGNORED_BAD_PRACTICE	1	
WA_AWAIT_NOT_IN_LOOP	1	
Total	22	

Test Report

<u>Tests</u>							
Job	Success		Failed		Skipped		Total
	#	%	#	%	#	%	#
 Eden-ActiveWorlds	127	100%	0	0%	0	0%	127
 Eden-Eden	266	100%	0	0%	0	0%	266
 Eden-MondiDinamiciWebservice	155	100%	0	0%	0	0%	155
 Eden-Palm	20	100%	0	0%	0	0%	20
Total	568	100%	0	0%	0	0%	568

Test Result

0 failures (±0) , 2 skipped (±0)

1,513 tests (±0)

Module	Fail	(diff)	Total	(diff)
org.apache.httpcomponents:httpClient	0		582	
org.apache.httpcomponents:httpClient-cache	0		920	
org.apache.httpcomponents:httpmime	0		11	

Test Code Coverage

Packages

[All](#)

[net.sourceforge.cobertura.ant](#)
[net.sourceforge.cobertura.check](#)
[net.sourceforge.cobertura.cover](#)
[net.sourceforge.cobertura.instru](#)
[net.sourceforge.cobertura.merg](#)
[net.sourceforge.cobertura.repor](#)
[net.sourceforge.cobertura.repor](#)
[net.sourceforge.cobertura.repor](#)

All Packages

Classes

[AntUtil](#) (88%)
[Archive](#) (100%)
[ArchiveUtil](#) (80%)
[BranchCoverageData](#) (N/A)
[CheckTask](#) (0%)
[ClassData](#) (N/A)

Coverage Report - All Packages

Package	# Classes	Line Coverage	Branch Coverage	Complexity
All Packages	55	75% 1625/2179	64% 472/738	2.319
net.sourceforge.cobertura.ant	11	52% 170/330	43% 40/94	1.848
net.sourceforge.cobertura.check	3	0% 0/150	0% 0/76	2.429
net.sourceforge.cobertura.coveragedata	13	N/A N/A	N/A N/A	2.277
net.sourceforge.cobertura.instrument	10	90% 460/510	75% 123/164	1.854
net.sourceforge.cobertura.merge	1	86% 30/35	88% 14/16	5.5
net.sourceforge.cobertura.reporting	3	87% 116/134	80% 43/54	2.882
net.sourceforge.cobertura.reporting.html	4	91% 475/523	77% 156/202	4.444
net.sourceforge.cobertura.reporting.html.files	1	87% 39/45	62% 5/8	4.5
net.sourceforge.cobertura.reporting.xml	1	100% 155/155	95% 21/22	1.524
net.sourceforge.cobertura.util	9	60% 175/291	69% 70/102	2.892
someotherpackage	1	83% 5/6	N/A N/A	1.2

Report generated by [Cobertura](#) 1.9 on 6/9/07 12:37 AM.

Ref: <http://cobertura.sourceforge.net/sample/>

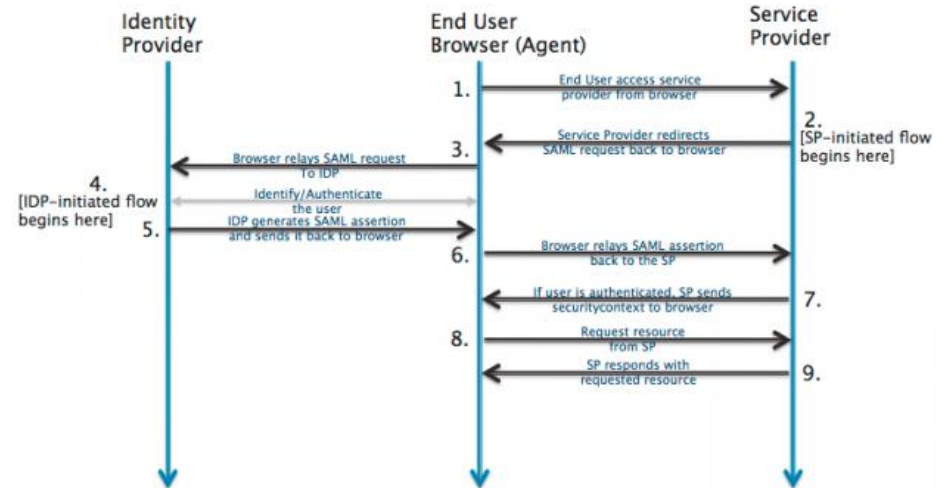
Artifact Uploader Plugins

- <https://www.jenkins.io/doc/developer/publishing/artifact-repository/>
AWS/Azure/Google Cloud/Heroku
- Dropbox
- Websphere
- FTP/SCP/SSH
- [Artifactory](#) - allows your build jobs to deploy artifacts automatically to Artifactory and have them linked to the build job that created them ([video](#), 1 min)

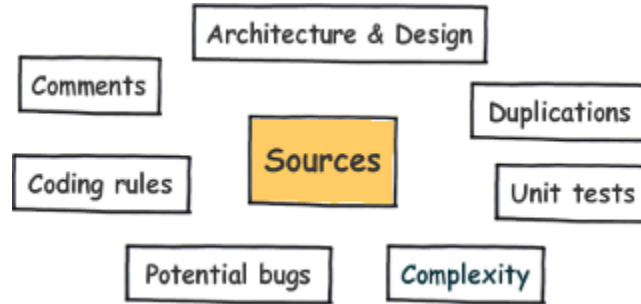


Security Management

- Security Realm
 - SAML (SSO)
 - LDAP
 - Jenkins's own user database
 - Delegate to servlet container
- Authorization
 - Anyone can do anything
 - Logged-in users can do anything
 - Matrix-based security
 - Project-based Matrix Authorization Strategy
 - Legacy mode



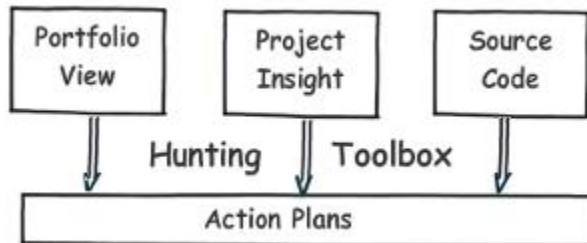
SonarQube



[SonarQube](#) is an open source platform for continuous inspection of code quality. It has an efficient way of navigating among high-level view, dashboard, and [Analysis](#) (1 min)

This enables to quickly uncover projects and / or components that are in Technical Debt to establish action plans.

- SonarQube™ software (previously known as “Sonar”) is an open source project written in Java.
- Current version: SonarQube Server 2026.4 (July 2026); the Community Build is free, under LGPL v3
- <https://www.sonarsource.com/products/sonarqube/>



CONTINUOUS DELIVERY & INTEGRATION

Definitions

- **Agile Development Process:**

*"An **iterative and incremental** (evolutionary) approach to software development which is performed in a **highly collaborative manner by self-organizing teams** within an effective governance framework with "just enough" ceremony that produces **high quality solutions in a cost effective and timely manner which meets the changing needs of its stakeholders.**"*

– Scott Ambler

Definitions Cont.

- **Agile Manifesto**: Principles that outline the Agile Process
- **Continuous Delivery**: New software development process that aims to build software that is always ready to be deployed into production
- **Continuous Integration**: Technique where every developer on a project must integrate their work daily (continuously) with every other developer

Background/History

- [Agile Manifesto](#) was written in 2001
 - **Martin Fowler**
 - **Kent Beck**
 - [XP Explained](#)
 - They both wrote about Continuous Integration around the year 1999

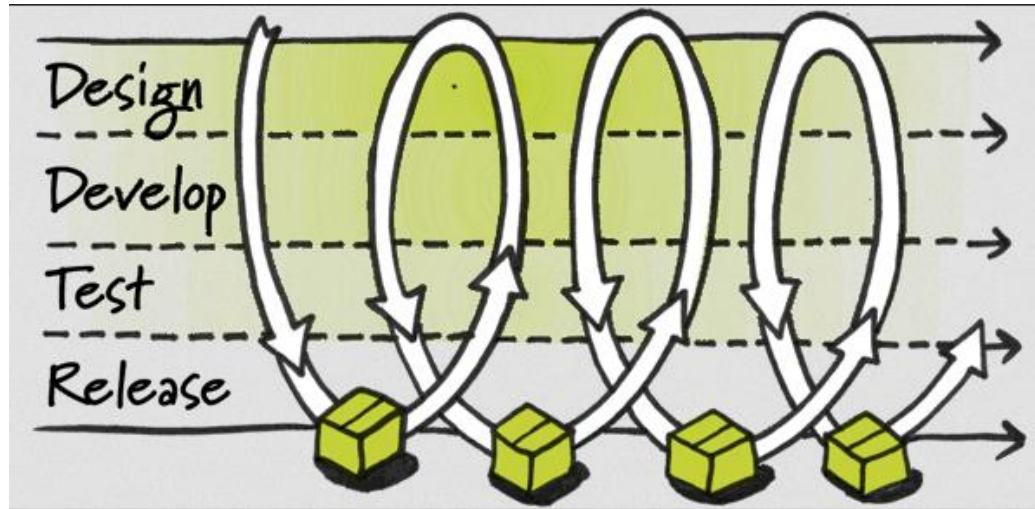


Manifesto for Agile Software Development

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

Continuous Delivery

- Recall: Build software that is always ready to be deployed into production



Continuous Delivery cont.

- 1st principle of the Agile Manifesto:
 - “Our highest priority is to satisfy the customer through early and **continuous delivery** of valuable software.”
- Goal: Release to production **more often**
 - ~~Monthly~~
 - Weekly?
 - **Daily**

Continuous Delivery cont.

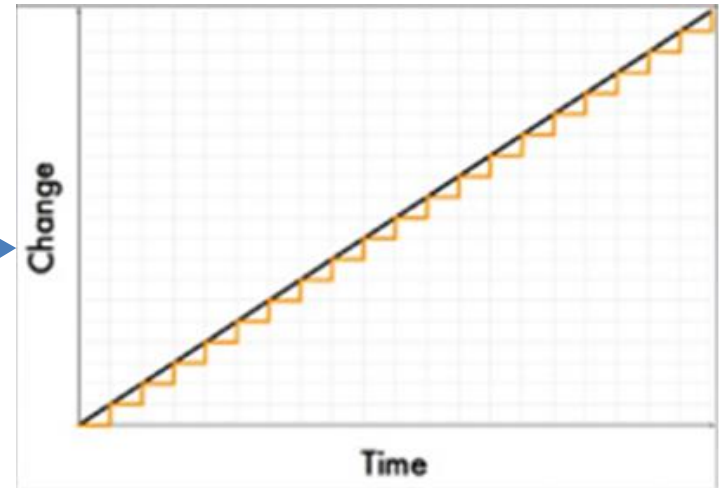
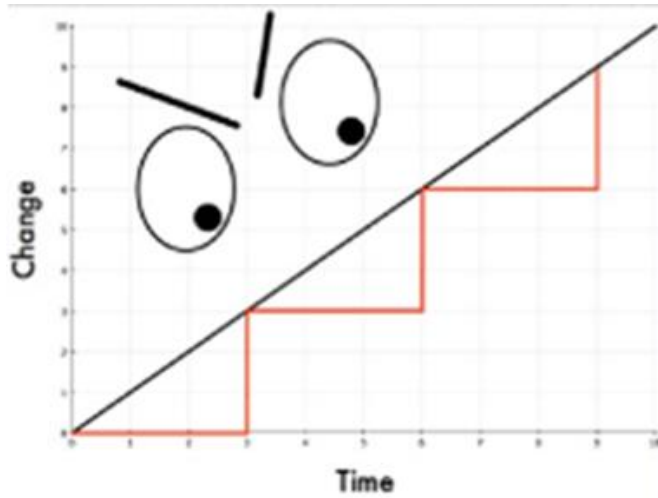
- Pros: Receive many benefits from the Agile process as well as others
 - Build the right product
 - Constant feedback from customers
 - Write thorough acceptance tests
 - Earlier benefits
 - Get product out before competitors
 - Find bugs earlier

Continuous Delivery cont.

- Ability to react quickly to change
 - Not a huge amount of time and money can be abandoned to adopt new requirements
- Save money and time
 - Save money and time if project goes bad
 - Save money and time with automation
 - As deploying to production becomes easier, more time can be spent developing valuable features

Continuous Delivery cont.

- Pros cont:
 - Reliability



Practice Continuous Delivery

- Basics to start practicing Continuous Delivery
 - **Configuration Management (CM)**
 - What needs to be kept under CM?
 - Everything! (Infrastructure as Code)
 - Code
 - Tests
 - Configuration Files
 - Build Scripts
 - Environments
 - Documentation
 - Etc.

CM Branching

- Stay away from branching except in special cases
 - Branch on releases
 - Horror story example
- You must always check into the trunk!
 - Otherwise you're not continuously integrating

CM Branching

- What if your project team is releasing to production every week, but you're working on a feature that will take longer than the release cycle to implement?
- First option:
 - Gradually release feature into production
- Second option:
 - Feature toggle

Managing Environments

- Must have multiple environments when developing software
- Need to be able to duplicate environments with ease
- Environments configurations to take consideration of:
 - Operating systems including their framework and settings
 - Packages needed to be installed for the application to function properly
 - Network settings

Continuous Integration

- Vital step when practicing CD
- Recall: To practice CI, every developer on a project must integrate their work daily with every other developer
 - Everyone on the team needs to practice this for it to work
- Continuous integration is **not a tool**, but a technique
 - But there are many open source tools to help practice CI

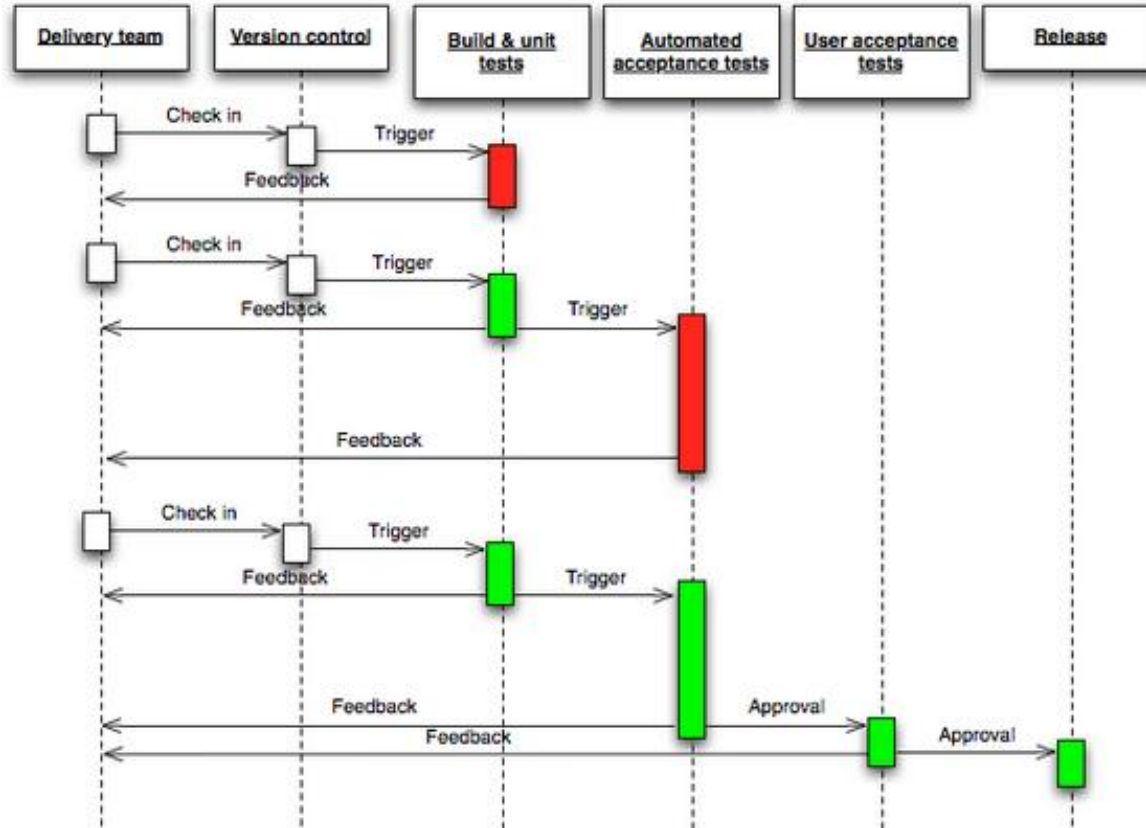
Continuous Integration

- Things a CI server can do:
 - After you check-in code it can:
 - Run build/test scripts
 - Notify developers of a failed build or failed tests
 - Can show all past check-ins and if they failed or passed all the tests
 - Can keep track of multiple projects
 - Plus much more!!

Deployment Pipeline

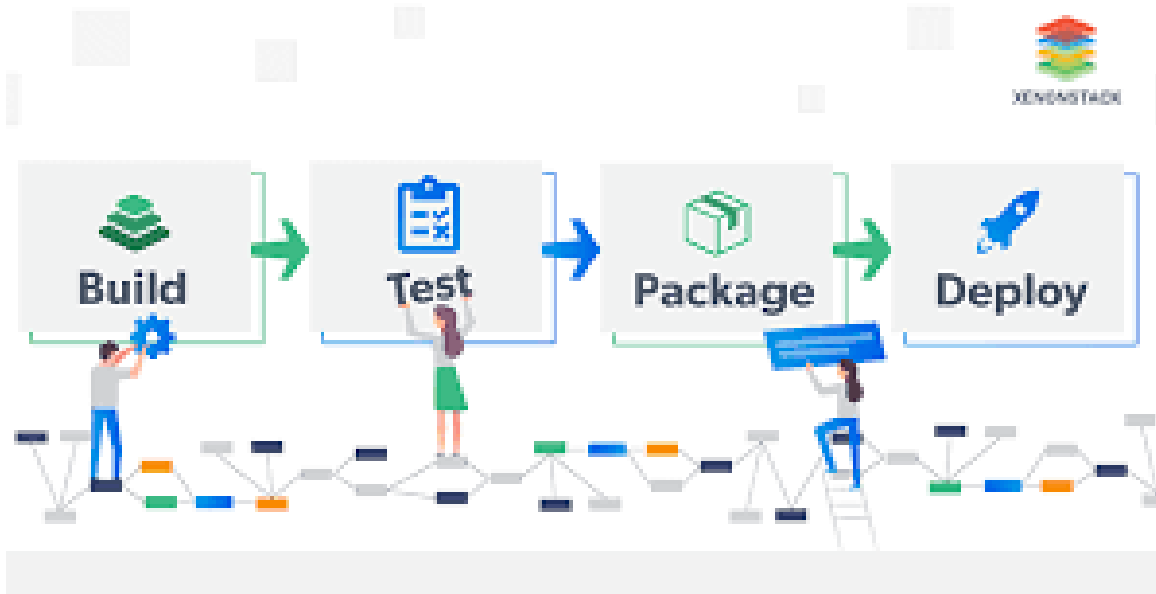
- Heart of Continuous Delivery
- Will give immediate feedback for how ready your software is for production
- Automate everything(unit tests, acceptance tests, performance tests,...) to the point of just being able to click a button to deploy to production

Deployment Pipeline cont.

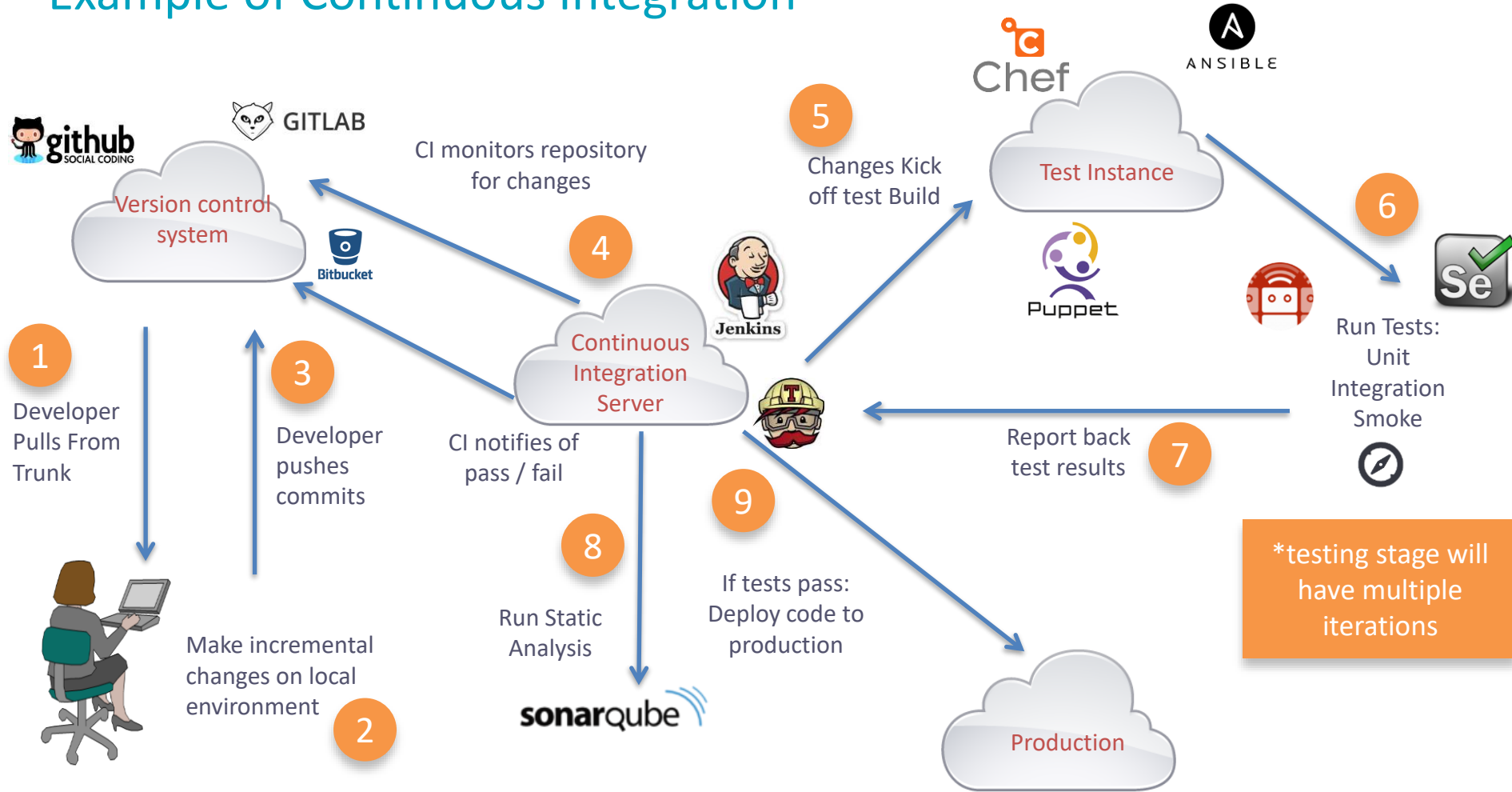


Continuous Delivery Metaphor

- [Continuous Delivery Assembly Line Metaphor](#)
(2 mins)



Example of Continuous Integration



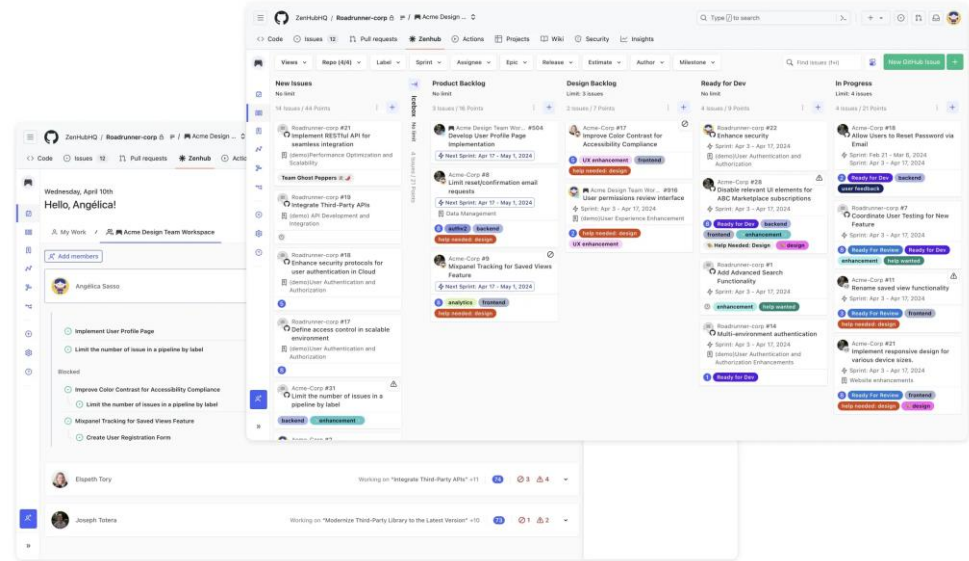


GITHUB PROJECTS IN SOFTWARE DEVELOPMENT METHODOLOGY

GitHub Projects

- GitHub Projects is GitHub's project management and planning tool integrated directly with repositories.
- It provides kanban-style boards, timelines, and automation features for tracking issues, pull requests, and tasks.
- Used widely in agile, DevOps, and enterprise workflows to connect code with project planning.
- Single Source of Truth: Developers, PMs, and stakeholders can see tasks, progress, and code in one place.
- Bridges Development & Management: Links planning (roadmaps, priorities) directly to execution (commits, issues, PRs).
- Supports Modern Methodologies: Fits agile (sprints, backlogs), kanban (continuous flow), and hybrid enterprise methods.

Projects: Core Features



- Boards:** Kanban-style columns (To Do, In Progress, Done).
- Timelines:** Gantt-like views for roadmap planning.
- Custom Fields:** Add priority, status, points, assignee, etc.
- Issue & PR Integration:** Automatically sync work items with repository activity.
- Automation:** Rules to auto-move cards when issues close, PRs merge, or labels change.
- Insights:** Charts for burndown, velocity, and bottleneck tracking.

GitHub Projects in Agile / DevOps

- **Sprint Planning:** Define goals, assign issues, track completion.
- **Continuous Delivery Pipelines:** Link deployments to project tracking.
- **Standups:** Use boards as the daily visual artifact of progress.
- **Retrospectives:** Review project insights (e.g., which tasks often got blocked).

<https://www.youtube.com/watch?v=vqbcNXtHgvg>



<https://github.com/Lisa-Vanderschuit>

In-Class– Jenkins Assignment

In-Class: Getting started with the Guided Tour

- <https://www.jenkins.io/doc/pipeline/tour/getting-started/>
 - ["Create your first Pipeline"](#)
 - ["Run multiple steps"](#)
 - ["Defining execution environments"](#)
 - ["Using environment variables."](#)
 - ["Recording tests and artifacts"](#)
 - ["Cleaning up and notifications"](#)
 - ["Deployment"](#)

Jenkins 2.568 LTS needs Java 21 or 25. The tour page still says 17 or 21; use 21.

Homework #4

For Homework #4,

A) Integrate Jenkins with your Project GitHub repository

- <https://plugins.jenkins.io/git/>
- <https://www.jenkins.io/doc/book/installing/macos/>
- <https://www.jenkins.io/doc/book/installing/windows/>

B) Implement at least one build trigger (E.g., [Webhooks](#))

C) Quickstart for Github Projects

- Complete each of the following steps:
 - [Introduction](#)
 - [Prerequisites](#)
 - [Creating a project](#)
 - [Setting your project description and README](#)
 - [Adding issues to your project](#)
 - [Adding draft issues to your project](#)
 - [Adding an iteration field](#)
 - [Creating a field to track priority](#)
 - [Grouping issues by priority](#)
 - [Saving the priority view](#)
 - [Adding a board layout](#)
 - [Configure built-in automation](#)

Submit a Word Document, with
Team Members Contribution
Description of integration done, with Screen shots,
etc..

Reading Assignment

- Fowler Ch. 5 Concurrency
- Newman Ch. 7 Testing

References

- https://www.whitehouse.gov/sites/default/files/whitehouse_files/microsites/ostp/NSTC/preparing_for_the_future_of_ai.pdf
- <http://www.slideshare.net/DennysHsieh/continuous-integration-jenkinshudson-7232207>
- http://static.hjug.org/present/Java_Build_Tool_Comparison.ppt
- <http://www.djaa.com/>
- <http://www.djaa.com/kanban-successful-evolutionary-change-your-technology-business-0>

References

- Agile Modeling (AM) Home Page: Effective Practices for Modeling and Documentation. (n.d.). *Agile Modeling (AM) Home Page: Effective Practices for Modeling and Documentation*. Retrieved April 15, 2013, from <http://www.agilemodeling.com/>
- Waters, K. (n.d.). 7 Reasons why Continuous Delivery needs to be a BUSINESS initiative | All About Agile. *All About Agile | Agile Development Made Easy!*. Retrieved April 16, 2013, from <http://www.allaboutagile.com/7-reasons-why-continuous-delivery-needs-to-be-a-business-initiative/>
- Martin Fowler. (n.d.). *Martin Fowler*. Retrieved April 16, 2013, from <http://martinfowler.com/>
- Shore, James. "James Shore: Continuous Integration on a Dollar a Day." *James Shore: The Art of Agile*. N.p., n.d. Web. 19 Mar. 2013. <<http://www.jamesshore.com/Blog/Continuous-Integration-on-a-Dollar-a-Day.html>>.

References

- Agile ALM for Software Development from ThoughtWorks Studios. (n.d.). *Agile ALM for Software Development from ThoughtWorks Studios*. Retrieved April 17, 2013, from <http://www.thoughtworks-studios.com/>
- Puppet Labs. (n.d.). *Puppet Labs: IT Automation Software for System Administrators*. Retrieved April 16, 2013, from <https://puppetlabs.com/>
- Configuration management software | Open source configuration management - CFEngine - Distributed Configuration Management. (n.d.). *Configuration management software | Open source configuration management - CFEngine - Distributed Configuration Management*. Retrieved April 17, 2013, from <http://cfengine.com/>
- Allspaw, J. (n.d.). Ops Meta-Metrics: The Currency You Pay For Change. *Upload & Share PowerPoint presentations and documents*. Retrieved April 17, 2013, from <http://www.slideshare.net/jallspaw/ops-metametrics-the-currency-you-pay-for-change>

References

- Manifesto for Agile Software Development. (n.d.). *Manifesto for Agile Software Development*. Retrieved April 14, 2013, from <http://agilemanifesto.org>
- Humble, Jez, and David Farley. *Continuous delivery*. Upper Saddle River, NJ: Addison-Wesley, 2011. Print.

SAN JOSÉ STATE UNIVERSITY *powering* SILICON VALLEY